

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.10.11	完成日期	2024.10.11

第一题

1. 实验题目

All pairs shortest path 的 Dijkstr 算法

2. 实验目的

使用 Dijkstr 算法求解 All pairs shortest path 问题

3. 算法设计

1. 创建能够表示带权有向图的矩阵 arc、顶点数 vexNum、边数 arcNum、表示顶点是否已经被查过的数组 s
2. 根据输入, 对 arc 初始化, 对于任意 arc[i][j], 如果 i=j 则为 0 表示到自己没有代价; 如果没有边则为无穷大; 如果有边表示权重
3. 利用数组 s, 可在进行 Dijkstr 算法时查找还没遍历过的结点中路径最小的
4. Dijkstr 算法: ①从第一个顶点开始进行遍历, 即第一次访问第一个顶点, 若存在询问的起始顶点到第一个顶点的有向边, 则给一维数组 dist 赋值, 为 arc 对应的权重 ②对标记访问过结点的 s, 第一次只有起始结点为标记 ③进行核心的更新部分: 寻找当前所有结点, 未被访问的结点, 且具有最小权重的结点; 通过该结点进行扩展, 对为访问过的结点, 通过该结点能使 dist 变小则更新, 即原来的路径时 dist[i], 通过当前的最小值结点, 则路径变为 dist[min]+arc[min][i]; 同时标记更新; 循环操作。

4. 程序运行与测试

1. 有向图的类

```
const int MAX_V_NUM = 100;

class EdgeGraph
{
private:
    int arc[MAX_V_NUM][MAX_V_NUM] = { INT_MAX }; //有向边的权重 最大int表示权重无穷
    int vexNum, arcNum; //图的顶点数和边数
    int s[MAX_V_NUM] = { 0 }; //存储顶点是否被查找过
public:
    EdgeGraph(int vNum, int eNum); //构造函数
    void Dijkstra(int v1, int v2);
    int findMinDist(int* dist); //在dist中查找 未被找过的（即s[i]为0）的最小值元素
};
```

2. 构造距离矩阵

```
EdgeGraph::EdgeGraph(int vNum, int eNum)
{
    int u, v, w;
    vexNum = vNum;
    arcNum = eNum;

    //arc初始化
    for (int i = 1; i <= vexNum; i++)
    {
        for (int j = 1; j <= vexNum; j++)
        {
            if (i != j)
                arc[i][j] = INT_MAX;
            else
                arc[i][j] = 0;
        }
    }
    for (int i = 0; i < arcNum; i++)
    {
        cin >> u >> v >> w; //依次输入边的起点编号，终点编号和权值
        if (w < arc[u][v])
            arc[u][v] = w; //防止重复的边但权值不同 取较小者
    }
}
```

3. Dijkstra 算法

```

void EdgeGraph::Dijkstra(int start,int end)
{
    int dist[MAX_V_NUM] = { INT_MAX };
    int num = 0; //记录顶点数
    int min; //记录最小值

    for (int i = 1; i <= vexNum; i++)
        dist[i] = arc[start][i];

    //s数组的初始化
    for (int i = 1; i <= vexNum; i++)
        s[i] = 0;
    s[start] = 1; //顶点放入集合s。
    num = 1;
    while (num < vexNum) //当顶点数小于图的顶点数
    {
        min = findMinDist(dist); //找到最小值
        if (min == 0) //找不到最小值
            break;
        for (int i = 1; i <= vexNum; i++)
        {
            //更新dist
            if ((s[i] == 0) && dist[min] != INT_MAX && arc[min][i] != INT_MAX && (dist[min] + arc
            [min][i] < dist[i]))
                //找到更短的距离，防止距离为无穷的情况
                dist[i] = dist[min] + arc[min][i];
        }
        s[min] = 1; //将新生成的点加入s
        num++;
    }
    if (dist[end] == INT_MAX)
        cout << "-1" << endl;
    else
        cout << dist[end] << endl;
}

```

4.算法使用的寻找最小值的函数

```

//找最小值
int EdgeGraph::findMinDist(int* dist)
{
    int min = INT_MAX;
    int index = 0;
    for (int i = 1; i <= vexNum; i++) //在dist数组中没有生成树（s[i]=0）的结点中查找
    {
        if (s[i] == 0)
        {
            if (dist[i] < min)
            {
                min = dist[i];
                index = i;
            }
        }
    }
    return index;
}

```

5. 实验总结与心得

1. Dijkstra 算法在求解任意两个顶点之间的最小路径时，需要多次重复使用算法，这一点的效率较 Floyd 算法效率差
2. Dijkstra 算法的实现：①从第一个顶点开始进行遍历，即第一次访问第一个顶点，若存在询问的起始顶

点到第一个顶点的有向边，则给一维数组 `dist` 赋值，为 `arc` 对应的权重 ②对标记访问过结点的 `s`，第一次只有起始结点为标记 ③进行核心的更新部分：寻找当前所有结点，未被访问的结点，且具有最小权重的结点；通过该结点进行扩展，对为访问过的结点，通过该结点能使 `dist` 变小则更新 3.本人在做实验题过程在，在更新时的条件判断上耗时较多，尤其是更新结点时的长度判断，因此需要合理的初始化距离矩阵，合理使用 `dist` 和 `s` 等数组进行判断。

第二题

1. 实验题目

All pairs shortest path 的 Floyd 算法

2. 实验目的

使用 Floyd 算法求解 All pairs shortest path 问题

3. 算法设计

1. Floyd 类里的成员：1.顶点数 2.边数 3.邻接矩阵 4.Floyd 的算法 5.输出结果函数
2.Floyd 算法核心：1.初始化邻接矩阵，其中注意对角线元素和无边的无穷大值 2.输入带权的有向边 3.进行三层循环，不断缩短从 `i` 到 `j` 之间的距离，即每次设法通过 `k` 来缩短 `i` 与 `j` 的距离，判断 `arc[i][j]` 与 `arc[i][k]+arc[k][j]` 的大小关系 3.输出，题目要求不存在有向边则输出为-1

4. 程序运行与测试

1. Floyd 类成员

```

class Floyd
{
public:
    int vexNum;//顶点数
    int arcNum;//边数目
    int arc[MAX_V_NUM][MAX_V_NUM]; //邻接矩阵
    Floyd(int vexNum, int arcNum); //初始化邻接矩阵 Floyd算法
    void FloydPrint(int begin, int end); //输出结果的函数
};

```

2. Floyd 算法

```

Floyd::Floyd(int vexNum, int arcNum)
{
    for (int i = 1; i <= vexNum; i++) //初始化距离矩阵 无穷
    {
        for (int j = 1; j <= vexNum; j++)
        {
            if (i != j)
                arc[i][j] = INT_MAX;
            else
                arc[i][j] = 0;
        }
    }

    int u, v, w;
    for (int i = 0; i < arcNum; i++) //输入带权有向边
    {
        cin >> u >> v >> w;
        if (w < arc[u][v])
            arc[u][v] = w; //防止重复的边但权值不同 取较小者
    }

    for (int k = 1; k <= vexNum; k++)
        for (int i = 1; i <= vexNum; i++)
            for (int j = 1; j <= vexNum; j++)
                if (arc[i][k] != INT_MAX && arc[k][j] != INT_MAX && arc[i][k] != 0 &&
                    arc[k][j] != 0 && arc[i][k] + arc[k][j] < arc[i][j])
                    //从i到j 通过k可以使得路径更短
                    arc[i][j] = arc[i][k] + arc[k][j]; //更新权重(长度)
}

```

5. 实验总结与心得

1. Floyd 算法核心：1.初始化邻接矩阵，其中注意对角线元素和无边的无穷大值 2.输入带权的有向边 3.进行三层循环，不断缩短从 i 到 j 之间的距离，即每次设法通过 k 来缩短 i 与 j 的距离，判断 arc[i][j] 与

$\text{arc}[i][k] + \text{arc}[k][j]$ 的大小关系

2. Floyd 算法较 Dijkstra 算法简单, 只需要通过三层循环进行判断即可, 在这道题中比较适合但如果只需要求一次结果的话, Dijkstra 算法可能更合适

第三题

1. 实验题目

Robot

2. 实验目的

Karell Incorporated has designed a new exploration robot that has the ability to explore new terrains, this new robot can move in all kinds of terrain, it only needs more fuel to move in rough terrains, and less fuel in plain terrains. The only problem this robot has is that it can only move orthogonally, the robot can only move to the grids that are at the North, East, South or West of its position.

The Karell's robot can communicate to a satellite dish to have a picture of the terrain that is going to explore, so it can select the best route to the ending point, The robot always choose the path that needs the minimum fuel to complete its exploration, however the scientist that are experimenting with the robot, need a program that computes the path that would need the minimum amount of fuel. The maximum amount of fuel that the robot can handle is 9999 units.

The Terrain that the robot receives from the satellite is divided into a grid, where each cell of the grid is assigned to the amount of fuel the robot would need to pass through that cell. The robot also receives the starting and ending coordinates of the exploration area.

3. 算法设计

1. 使用 Dijkstra 算法: 1. 从开始结点触发出发, 每次选择已有结点 2. 访问东西南北四个结点, 找出访问后可以改变的最小值, 更新 3. 其中访问某一访问的节点时, 如果结点存在且未被访问则进行更新, 判断是否可以缩短路径则更新, 判断是否为当前找的最小值则更新 4. 只有循环遍历完当前已经访问的所有结点, 才能将该最小值结点更新为已经访问结点

4. 程序运行与测试

1. 输入与数组创建

```

//输入
int row, col;
cin >> row >> col;
vector<vector<int>> Map(row + 1, vector<int>(col + 1)); //第零行 第零列不用
for (int i = 1; i <= row; i++)
    for (int j = 1; j <= col; j++)
        cin >> Map[i][j];

int start_x, start_y, end_x, end_y;
cin >> start_x >> start_y >> end_x >> end_y;

vector<vector<int>> dis(row + 1, vector<int>(col + 1, INT_MAX)); //距离
vector<vector<int>> visit(row + 1, vector<int>(col + 1, 0)); //访问 0: 未被访问
vector<pair<int, int>> visited; //已经访问过的结点集合

```

2.初始化

```

//初始化
dis[start_x][start_y] = Map[start_x][start_y];
visit[start_x][start_y] = 1;
visited.push_back(make_pair(start_x, start_y));

```

3.Dijkstra 算法

```

while (!visit[end_x][end_y]) //如果终点没有被访问，则继续
{
    int min_x = -1, min_y = -1; //待扩展的突破口
    int min_weight = INT_MAX;

    for (int i = 0; i < visited.size(); i++)
    {
        int x = visited[i].first;
        int y = visited[i].second;
        //向北
        if (x - 1 >= 1 && !visit[x - 1][y])
        {
            if(dis[x][y] + Map[x - 1][y] < dis[x - 1][y])
                dis[x - 1][y] = dis[x][y] + Map[x - 1][y];
            if (dis[x - 1][y] < min_weight)
            {
                min_weight = dis[x - 1][y];
                min_x = x - 1;
                min_y = y;
            }
        }
    }
}

```



```

//向东
if (y + 1 <= col && !visit[x][y + 1])
{
    if(dis[x][y] + Map[x][y + 1] < dis[x][y + 1])
        dis[x][y + 1] = dis[x][y] + Map[x][y + 1];
    if (dis[x][y + 1] < min_weight)
    {
        min_weight = dis[x][y + 1];
        min_x = x;
        min_y = y + 1;
    }
}

if (min_x!=-1) //如果找到了新的节点，才更新
{
    visit[min_x][min_y] = 1;
    visited.push_back(make_pair(min_x, min_y)); //添加到访问节点列表
}
}

```

5. 实验总结与心得

1. 该算法难度不大
2. 但本人在每次对一个方位的结点进行判断时搞错判断条件，在进行访问时更新 dis 和更新当前已找的最小值这两步要分开
3. 要熟悉使用 visited 数组来帮助快速解题
4. Dijkstra 算法：1.从开始结点触发出发，每次选择已有结点 2.访问东西南北四个结点，找出访问后可以改变的最小值，更新 3.其中访问某一访问的节点时，如果结点存在且未被访问则进行更新，判断是否可以缩短路径则更新，判断是否为当前找的最小值则更新